

以下使用从 0 开始的左闭右开区间 $[l, r)$ 。记 $\text{LCP}_S(x, y)$ 为原串中从 x, y 开始的两个后缀的 LCP。若 $x < y$ ，它们在询问子串中的 LCP 为

$$\min\{\text{LCP}_S(x, y), r - y\}. \quad (1)$$

先求原串的后缀数组 sa、排名 rk 和相邻 LCP：

$$\text{lc}[i] = \text{LCP}_S(\text{sa}[i], \text{sa}[i + 1]) \quad (0 \leq i < n - 1).$$

把相邻的 SA rank 连成一条链，第 i 条边的权值为 $\text{lc}[i]$ 。按边权从大到小合并，每个连通块用 `set` 维护其中的文本位置。

合并两个块时，枚举较小集合中的位置 p ，只在较大集合中找它的前驱和后继 q ，记录候选

$$x = \min(p, q), \quad y = \max(p, q), \quad d = \text{LCP}_S(p, q).$$

候选不会漏掉答案。对任意询问答案对应的长度为 k 的字符串，在询问内的出现位置中取相邻的一对 $p < q$ 。当包含它们的两个块第一次合并时，两块中不可能还有位置落在 (p, q) 内，所以这对位置一定会被前驱或后继找到。

启发式合并保证每个位置只被枚举 $O(\log n)$ 次，因此候选数为 $O(n \log n)$ ，构造时间为 $O(n \log^2 n)$ 。相同权值的边需要逐条先查询再合并。

候选 (x, y, d) 对 $[l, r)$ 的贡献为

$$\min(d, r - y),$$

前提是 $x \geq l$ 且 $y < r$ 。令 $z = y + d$ ，则

$$\min(d, r - y) = \begin{cases} d, & z \leq r, \\ r - y, & z > r. \end{cases}$$

将候选按 x 分桶，从大到小扫描 l ，此时数据结构中恰好包含所有 $x \geq l$ 的候选。用两棵 Fenwick 树分别维护：

- $z \leq r$ 时最大的 d ；
- $z > r$ 时最小的 y ，贡献为 $r - y$ ，并检查 $y < r$ 。

两部分取最大值即为 H 。若答案相同，还要选择字典序更小的重复串。给候选附上

$$w = \min\{\text{rk}[x], \text{rk}[y]\},$$

并在贡献相同时取最小的 w 即可。

取最优候选的一个端点 p 。用 lc 的 RMQ 和两次二分，求出所有与 p 的 LCP 不小于 H 的 SA rank 区间 $[A, B]$ 。

对 $\text{rk}[0], \text{rk}[1], \dots, \text{rk}[n - 1]$ 建立划分树（主席树），支持在一个文本位置区间中取值域 $[A, B]$ 内最小的若干 rank。令 $R = r - H$ 。

- 若 $\text{rk}[R] \in [A, B]$ ，位置 R 的询问后缀长度恰为 H ，必然排在最前；第二个位置取 $[l, R)$ 中属于 $[A, B]$ 的最小 rank。
- 否则在 $[l, R + 1)$ 中取属于 $[A, B]$ 的最小两个 rank。

第二种情况下所有可选后缀都至少还有 $H + 1$ 个字符。若其中两个完整后缀的 LCP 大于 H ，询问答案也会大于 H ，所以按完整后缀的 rank 排序是正确的。

单组时间复杂度为 $O(n \log^2 n + m \log n)$ ，空间复杂度为 $O(n \log n + m)$ 。

